

Proyecto Análisis Exploratorio de Datos

María de los Ángeles Díaz Castro, Irene Barba La Orden, Ana Blasco Vega

* Correspondence:

Simple Summary: Preparación y análisis exploratorio del tráfico diario de carreteras en Valencia 2025, obtenido de la página oficial de la Generalitat Valenciana.

Abstract: Este proyecto tiene como objetivo analizar el tráfico en Valencia durante 2025, utilizando datos del dataset público proporcionado por la Generalitat Valenciana. Se consideran 18 variables, de las que se incluyen el número de vehículos que transitan por cada tramo, el método de captación o las coordenadas iniciales y finales de cada registro, entre otras. En total, se han recopilado 15.235 observaciones asociadas a distintos momentos en los que se registró tráfico. Los datos se reestructuraron para facilitar un análisis exploratorio previo, identificando patrones de circulación y posibles anomalías, tratando de ofrecer información valiosa para mejorar la gestión del tráfico y la planificación de las carreteras en Valencia.

1. Introducción.

El tráfico de vehículos va más allá de los números, muestra cómo Valencia se mueve, organiza y enfrenta cada día sus desafíos de desplazamiento. Este proyecto se centra en analizar la intensidad del tráfico en las carreteras de la Comunidad Valenciana durante 2025, a partir de datos públicos proporcionados por la Generalitat Valenciana. A través de un análisis exploratorio de 15.235 observaciones, se busca descubrir patrones de circulación, identificar posibles anomalías y generar información que sea útil para la gestión del transporte. Las visualizaciones y estadísticas que se presentan a lo largo del documento permiten observar de manera clara cómo se mueve Valencia por sus carreteras, ofreciendo una visión tanto cuantitativa como práctica del tráfico en 2025.

2. Importación de los datos.

Para importar los datos, primero se descarga el fichero ([aquí](#)) y se verifica que los datos estén en el directorio adecuado. Es importante especificar el encoding (en nuestro caso es UTF-8), ya que, de lo contrario, algunos caracteres pueden no reconocerse correctamente. De esta forma, haciendo uso de la función de importación en R con los parámetros adecuados, obtenemos el dataset.

Para empezar, echaremos un pequeño vistazo a los datos para ver cuántas variables tenemos y de qué tipo son.

Citation: . Proyecto Análisis
Exploratorio de Datos. *Journal Not
Specified* **2024**, *1*, 0. <https://doi.org/>

Received:

Revised:

Accepted:

Published:

Copyright: © 2025 by the authors.
Submitted to *Journal Not Specified*
for possible open access publication
under the terms and conditions
of the Creative Commons Attri-
bution (CC BY) license ([https://
creativecommons.org/licenses/by/
4.0/](https://creativecommons.org/licenses/by/4.0/)).

```
glimpse(vehiculos)
```

```
## Rows: 15,235
```

```
## Columns: 18
```

```
## $ TRAM      <int> 10007, 10007, 10007, 10007, 10007, 10007, 10007, 10024, 1002~
```

```
## $ DIA      <chr> "07/03/2025", "08/03/2025", "09/03/2025", "10/03/2025", "11/~
```

```
## $ INTA      <dbl> 13.437, 6.871, 6.457, 14.053, 13.827, 14.218, 14.028, 13.774~
```

```
## $ INTAP      <dbl> 2.694, 505.000, 319.000, 2.985, 3.077, 3.134, 3.134, 2.793, ~
```

```
## $ INTD      <dbl> 13.437, 6.515, 6.243, 13.490, 13.394, 13.799, 13.779, 13.787~
```

```
## $ INTDP      <dbl> 2.674, 424.000, 290.000, 2.904, 2.864, 3.000, 3.065, 2.725, ~
```

```
## $ INTT      <dbl> 26.874, 13.386, 12.700, 27.543, 27.221, 28.017, 27.807, 27.5~
## $ INTTP     <dbl> 5.368, 929.000, 609.000, 5.889, 5.941, 6.134, 6.199, 5.518, ~
## $ TIPUSV    <chr> "Aut.", "Aut.", "Aut.", "Aut.", "Aut.", "Aut.", "Aut.", "Aut~
## $ TIPUSE    <chr> "Espiras", "Espiras", "Espiras", "Espiras", "Espiras", "Espi~
## $ PKE       <int> 700, 700, 700, 700, 700, 700, 700, 2400, 2400, 2400, 2400, 2~
## $ PKECOOR   <chr> "39,88828171;-0,1963450773", "39,88828171;-0,1963450773", "3~
## $ PKINI     <int> 0, 0, 0, 0, 0, 0, 0, 1650, 1650, 1650, 1650, 1650, 1650, 16~
## $ REFINI    <chr> "Ac. Vilavella des de BetxÃ", "Ac. Vilavella des de BetxÃ~
"~
## $ PKINICOOR <chr> "39,8805620167;-0,1927724982", "39,8805620167;-0,1927724982"~
## $ PKFI      <int> 1650, 1650, 1650, 1650, 1650, 1650, 1650, 5000, 5000, 5000, ~
## $ REFFI     <chr> "CV-223", "CV-223", "CV-223", "CV-223", "CV-223", "CV-223", ~
## $ PKFIC00R  <chr> "39,8964339871;-0,1969224331", "39,8964339871;-0,1969224331"~
```

Procedemos ahora a explicar qué significa cada columna. Las 18 variables del dataset se definen como sigue:

1. **TRAM:** Código del tramo al que se le asocia el tráfico.
Primer y segundo dígito: código CV- de la carretera; tres últimos dígitos: número de tramo.
2. **DIA:** Fecha de los datos.
3. **INTA:** Total de vehículos que transitan en el sentido ascendente de la kilometración de la carretera.
4. **INTAP:** Total de vehículos pesados que transitan en el sentido ascendente de la kilometración de la carretera.
5. **INTD:** Total de vehículos que transitan en el sentido descendente de la kilometración de la carretera.
6. **INTDP:** Total de vehículos pesados que transitan en el sentido descendente de la kilometración de la carretera.
7. **INTT:** Total de vehículos que pasan por el tramo analizado (este dato será la suma de los dos sentidos anteriores).
8. **INTTP:** Total de vehículos pesados que pasan por el tramo analizado (este dato será la suma de los dos sentidos anteriores).
9. **TIPUSV:** Tipo de vía (Autovía, Convencional o Desdoblada).
10. **TIPUSE:** Procedimiento de captura de tráfico (Espiras, Gomas Neumáticas, Cámaras o Radar).
11. **PKE:** Posición exacta, en metros, donde se ha captado el tráfico, tomando como referencia la placa de kilometraje anterior en sentido ascendente.
12. **PKECOOR:** Coordenadas geodésicas en latitud y longitud del punto donde se mide el tráfico.
13. **PKINI:** Posición inicial del tramo al que se asocia el tráfico, en metros, tomando como referencia la placa kilométrica anterior en sentido ascendente (tráfico invariante en todo el tramo).
14. **REFINI:** Ubicación del inicio del tramo al que se asocia el tráfico.
15. **PKINICOOR:** Coordenadas geodésicas en latitud y longitud del punto inicial del tramo al que se asocia el tráfico (tráfico invariante en todo el tramo).

16. **PKFI:** Posición final del tramo al que se asocia el tráfico, en metros,tomando como referencia la placa kilométrica anterior en sentido ascendente (tráfico invariante en todo el tramo).
17. **REFFI:** Ubicación del final del tramo al que se asocia el tráfico.
18. **PKFICOOR:** Coordenadas geodésicas en latitud y longitud del punto final del tramo al que se asocia el tráfico (tráfico invariante en todo el tramo).

Cuando hemos echado un vistazo al conjunto hemos podido observar que *DIA* es de tipo character, por lo que es necesario convertirla a formato de fecha.

```
vehiculos$DIA <- as.Date(vehiculos$DIA, format = '%d/%m/%Y')
class(vehiculos$DIA)
```

```
## [1] "Date"
```

Antes de proseguir modificando y explorando los datos vamos a considerar los valores faltantes haciendo uso de la función *aggr* del paquete *VIM*.

```
a <- aggr(vehiculos, sortVars = TRUE, plot = FALSE)
a$missings
```

```
##          Variable Count
## TRAM          TRAM    0
## DIA           DIA    0
## INTA          INTA    0
## INTAP         INTAP    0
## INTD          INTD    0
## INTDP         INTDP    0
## INTT          INTT    0
## INTTP         INTTP    0
## TIPUSV        TIPUSV    0
## TIPUSE        TIPUSE    0
## PKE           PKE     0
## PKECOOR       PKECOOR    0
## PKINI         PKINI    0
## REFINI        REFINI    0
## PKINICOOR     PKINICOOR    0
## PKFI          PKFI     0
## REFFI         REFFI     0
## PKFICOOR      PKFICOOR    0
```

Como puede apreciarse en el dataframe *missings*, que recuenta los valores faltantes de cada variable, nos encontramos en la feliz situación de no hallar ningún valor faltante, por lo que no será necesario llevar a cabo ningún proceso de imputación o similar.

3. Conversión de los datos a estructura Tidy.

La estructura tidy se caracteriza por tener cada variable en una columna, cada observación en una fila y nombres de columnas que sean representativos. Veamos que ocurre en nuestro dataset.

La columna *TRAM* contiene información del código CV- de la carretera (dos primeros dígitos) y del número de tramo(tres últimos dígitos). Por tanto, primero verificaremos que todos los valores de esta columna tengan cinco dígitos. Si es así, procederemos a dividirla en dos columnas separadas, que se llamarán *Código_CV* y *Num_tramo*.

```
all(length(vehiculos$TRAM == 5))
```

```
## [1] TRUE
```

```
vehiculos_tidy <- vehiculos %>%
  separate(col = TRAM, into = c('Codigo_CV', 'Num_tramo'), sep = 2)
```

Las variables INTA, INTAP, INTD, INTDP, INTT y INTTP contienen información sobre el tipo de vehículo, si es pesado o no, y el sentido del tráfico, ascendente o descendente. Con el fin de estructurar mejor estos datos, aplicamos *pivot_longer()*, generando dos nuevas variables, Sentido y Tipo_Vehiculo.

En la columna Tipo_Vehiculo, los valores en blanco se rellenan con *T*, que representa cualquier vehículo, mientras que *P* indica vehículos pesados.

Por otro lado, las columnas INTT y INTTP representan la intensidad total del tráfico, es decir, la suma de los vehículos que circulan en ambos sentidos (ascendente y descendente), diferenciando si son pesados o no. Dado que esta información puede obtenerse a partir de las demás variables, eliminaremos estas columnas y, en caso de ser necesario, podremos recalcular sus valores sumando las correspondientes columnas de sentido.

```
vehiculos_tidy <- vehiculos_tidy %>%
  pivot_longer(col = starts_with('INT'),
    names_to = c('Sentido', 'Tipo_Vehiculo'),
    names_pattern = '^INT([ADT]){1}(P?)$', values_to = 'Flujo_Vehiculos') %>%
  mutate(Tipo_Vehiculo = ifelse(Tipo_Vehiculo == 'P', 'P', 'T')) %>%
  filter(Sentido != 'T')
```

Del mismo modo, las columnas PKECOOR, PKINICOOR y PKFICOOR contienen las coordenadas geográficas, diferenciando entre la posición inicial, final y exacta. Para reorganizar esta información, aplicamos *pivot_longer()*, que genera dos nuevas columnas: Coordenadas, donde se indica el tipo de coordenada (inicial, final o exacta), y Valor_coordenada, que almacena su valor correspondiente.

Además, observamos que cada coordenada incluye la latitud y la longitud separadas por un punto y coma. Por ello, utilizamos la función *separate()* para dividir Valor_coordenada en dos columnas adicionales: Latitud y Longitud.

```
vehiculos_tidy <- vehiculos_tidy %>%
  pivot_longer(col = ends_with('COOR'), names_to = 'Coordenadas',
    names_pattern = '^PK(E|INI|FI)COOR$',
    values_to = 'Valor_coordenada') %>%
  separate(col = Valor_coordenada, into = c('Latitud', 'Longitud'), sep=';') %>%
  pivot_longer(col = c(Latitud, Longitud), names_to = 'Tipo_Coordenada',
    values_to = 'Valor_Coordenada')
```

En cuanto a las variables REFINI y REFFI, estas recogen las ubicaciones inicial y final del tramo donde se ha detectado el tráfico. Para reorganizar esta información, aplicamos *pivot_longer()*, generando dos nuevas columnas: Tipo_Ubicacion, que especifica si se trata de la ubicación inicial o final, y Ubicacion, que almacena el nombre de la vía o localización.

```
vehiculos_tidy <- vehiculos_tidy %>%
  pivot_longer(col = starts_with('REF'), names_pattern = '^REF(INI|FI)',
    names_to = 'Tipo_Ubicacion', values_to = 'Ubicacion')
```

Por último, las variables PKINI, PKFI y PKE indican la posición inicial, final y exacta donde se registró el tráfico, tomando como referencia la placa kilométrica anterior en sentido ascendente, suponiendo que el tráfico es invariante en todo el tramo.

Mediante la función `pivot_longer()`, se crean dos nuevas columnas: `Tipo_Posicion_Punto_Q` que especifica si la posición corresponde al punto inicial o final, y `Valor_Punto_Quilometrico`,¹⁵¹ que contiene el valor asociado a dicha posición.¹⁵²

```
vehiculos_tidy <- vehiculos_tidy %>%
  pivot_longer(col = matches('^PK(INI|FI|E)'), names_pattern = '^PK(INI|FI|E)',
               names_to = 'Tipo_Posicion_Punto_Quilometrico',
               values_to = 'Valor_Punto_Quilometrico')
```

La columna `Valor_Coordenada` contiene los valores de latitud y longitud, donde se utilizan comas como separador decimal. Por este motivo, R detecta la columna como tipo `character`, cuando en realidad debería ser numérica. Para solucionarlo, usamos la función `sub()` para reemplazar las comas por puntos, lo que nos permite convertir correctamente los valores a tipo numérico. Veámoslo:¹⁵³¹⁵⁴¹⁵⁵¹⁵⁶¹⁵⁷

```
str(vehiculos_tidy$Valor_Coordenada)
```

```
## chr [1:2193840] "39,88828171" "39,88828171" "39,88828171" "39,88828171" . . 458
```

```
vehiculos_tidy$Valor_Coordenada <- sub(',', '.', vehiculos_tidy$Valor_Coordenada)
vehiculos_tidy$Valor_Coordenada <- as.numeric(vehiculos_tidy$Valor_Coordenada)
str(vehiculos_tidy$Valor_Coordenada)
```

```
## num [1:2193840] 39.9 39.9 39.9 39.9 39.9 ...
```

Solo falta asegurarnos de que los nombres de las columnas sean representativos; para ello, utilizamos la función `rename()` y reemplazamos algunos valores abreviados por descripciones completas mediante la función `ifelse()`. Por ejemplo, en las variables relacionadas con coordenadas y ubicación, los códigos *INI*, *E* y *FI* se sustituyeron por *Inicial*, *Exacta* y *Final*; en *Sentido*, los valores *A* y *D* pasaron a *Ascendente* y *Descendente*; y en *Tipo_Vehiculo*, *T* y *P* se cambiaron por *Todos* y *Pesados*. En el caso de las coordenadas y ubicación, se hace de forma análoga.¹⁶⁰¹⁶¹¹⁶²¹⁶³¹⁶⁴¹⁶⁵¹⁶⁶

```
vehiculos_tidy <- vehiculos_tidy %>%
  dplyr::rename(Fecha = DIA, Tipo_Via = TIPUSV, Metodologia_Captacion= TIPUSE)

# Rescribimos algunos valores:
vehiculos_tidy$Sentido <-
  vehiculos_tidy$Sentido %>%
  replace(. == "A", "Ascendente") %>%
  replace(. == "D", "Descendente")

vehiculos_tidy$Tipo_Vehiculo <-
  vehiculos_tidy$Tipo_Vehiculo %>%
  replace(. == "T", "Todos") %>%
  replace(. == "P", "Pesados")

vehiculos_tidy$Coordenadas <-
  vehiculos_tidy$Coordenadas %>%
  replace(. == "INI", "Inicial") %>%
  replace(. == "E", "Exacta") %>%
  replace(. == "FI", "Final")

vehiculos_tidy$Tipo_Via <-
  vehiculos_tidy$Tipo_Via %>%
```

```
replace(. == "Aut.", "Autovia") %>%
replace(. == "Conv.", "Convencional") %>%
replace(. == "Desd.", "Desdoblada")
```

A continuación, se muestra un fragmento de las primeras filas del dataset, usando la función head():

```
head(vehiculos_tidy)
```

```
## # A tibble: 6 x 15
##   Codigo_CV Num_tramo Fecha      Tipo_Via Metodologia_Captacion Sentido
##   <chr>      <chr>    <date>    <chr>    <chr>                <chr>
## 1 10         007      2025-03-07 Autovia   Espiras              Ascendente
## 2 10         007      2025-03-07 Autovia   Espiras              Ascendente
## 3 10         007      2025-03-07 Autovia   Espiras              Ascendente
## 4 10         007      2025-03-07 Autovia   Espiras              Ascendente
## 5 10         007      2025-03-07 Autovia   Espiras              Ascendente
## 6 10         007      2025-03-07 Autovia   Espiras              Ascendente
## # i 9 more variables: Tipo_Vehiculo <chr>, Flujo_Vehiculos <dbl>,
## #   Coordinadas <chr>, Tipo_Coordenada <chr>, Valor_Coordenada <dbl>,
## #   Tipo_Ubicacion <chr>, Ubicacion <chr>,
## #   Tipo_Posicion_Punto_Quilometrico <chr>, Valor_Punto_Quilometrico <int>
```

4. Relación entre las variables

En este apartado estudiaremos si existe una relación lineal entre las variables de nuestro conjunto de datos. En primer lugar, comenzaremos con las variables numéricas:

```
vehiculos_num <- vehiculos_tidy[apply(vehiculos_tidy, is.numeric)]
```

Obteniendo las variables Flujo_Vehiculos, Valor_Coordenada y Valor_Punto_Quilometrico almacenadas en vehiculos_num.

A continuación, calculamos las matrices de correlación de Pearson y Spearman con la función cor() para determinar si existe o no una relación entre estas variables:

```
matrizcorr1 <- cor(vehiculos_num, use='complete.obs', method='pearson')
matrizcorr2 <- cor(vehiculos_num, use='complete.obs', method='spearman')
matrizcorr1
```

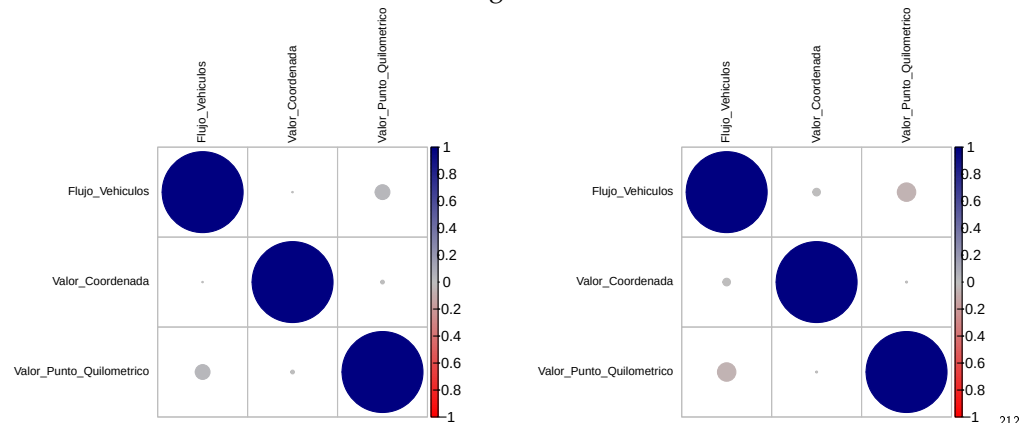
```
##           Flujo_Vehiculos Valor_Coordenada
## Flujo_Vehiculos      1.0000000000      0.0002915446
## Valor_Coordenada      0.0002915446      1.0000000000
## Valor_Punto_Quilometrico 0.0333949602      0.0021449420
##           Valor_Punto_Quilometrico
## Flujo_Vehiculos      0.033394960
## Valor_Coordenada      0.002144942
## Valor_Punto_Quilometrico 1.000000000
```

```
matrizcorr2
```

```
##           Flujo_Vehiculos Valor_Coordenada
## Flujo_Vehiculos      1.000000000      -0.0089011035
## Valor_Coordenada     -0.008901103      1.0000000000
## Valor_Punto_Quilometrico -0.051720007      -0.0005810784
##           Valor_Punto_Quilometrico
```

```
## Flujo_Vehiculos -0.0517200072
## Valor_Coordenada -0.0005810784
## Valor_Punto_Quilometrico 1.0000000000
```

Si nos fijamos en la matriz de Pearson, podemos ver que las correlaciones toman valores muy próximos a cero, lo que indica la ausencia de una relación lineal significativa entre VValor_Coordenada-Flujo_Vehiculos, Valor_Punto_Quilometrico-Flujo_Vehiculos y Valor_Punto_Quilometrico-Valor_Coordenada. Sin embargo, al analizar la matriz de Spearman, los valores siguen siendo cercanos a cero, aunque los signos cambian. Lo que indica que, en algunos casos, cuando una variable aumenta, la otra tiende a disminuir, aunque la relación no sea fuerte. Podemos verlo gráficamente:



Ahora estudiaremos la dependencia o independencia entre algunas variables categóricas: Tipo_Via-Metodologia_Captacion, Tipo_Via-Tipo_Vehiculo, Sentido-Tipo_Vehiculo. Comenzando con las dos primeras variables, calculamos el coeficiente V de Cramer (que está acotado entre 0 y 1):

```
tabla <- table(vehiculos_tidy$Tipo_Via, vehiculos_tidy$Metodologia_Captacion)
CramerV(tabla)
```

```
## [1] 0.07673791
```

Obteniendo 0.076, lo que nos indica que hay dependencia pero las variables prácticamente no se influyen entre sí. Veamos que ocurre con las otras variables:

```
tabla1 <- table(vehiculos_tidy$Tipo_Via, vehiculos_tidy$Tipo_Vehiculo)
CramerV(tabla1)
```

```
## [1] 0
```

```
tabla2 <- table(vehiculos_tidy$Sentido, vehiculos_tidy$Tipo_Vehiculo)
CramerV(tabla2)
```

```
## [1] 0
```

Obtenemos un valor del coeficiente para los pares de variables Tipo_Via-Tipo_Vehiculo y Sentido-Tipo_Vehiculo de 0; por tanto, ambos pares son independientes. Podríamos pensar que existe alguna dependencia entre el tipo de vehículo y el tipo de vía por la que se circula (autovía, convecional o desdoblada), pero observamos que no. En cambio, ya esperábamos que el sentido de circulación (ascendente o descendente) y el tipo de vehículo fueran completamente independientes ya que el tipo de vehículo no depende del sentido del trayecto (los vehículos pueden circular indistintamente en un sentido o en otro).

Data Availability Statement: Los datos usados en este estudio son públicos disponibles en [https://dadesobertes.gva.es/dataset/20b91b5a-c26d-4624-bcf5-b76a74b3ec88/resource/abe57190-d53d-4d41-9266-d7c2254621ae/download/intensidad_vehiculos_en_carretera_2025.csv]

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.

232

233

234